

THE ML/DL HANDBOOK — COMPANION SERIES · VOLUME V

From Linear Regression to Deep Neural Networks

The neuron, backpropagation, the optimizer family, and the vanishing-gradient story that quietly explains most of modern deep learning.

14

The Neuron & the Kink

15

Backprop & Vanishing

16

The Optimizer Family

17

Regularizing Deep Nets

4 chapters · every idea explained in plain English first · every figure measured, not drawn

Four chapters, each fixing where the last left off

14

The Neuron, and Why Depth Needs a Kink

Linear regression → neuron → why depth collapses without a nonlinearity

In plain English: a neuron is weighted voting, with a twist

15

Backpropagation, and the Vanishing Signal

The δ recursion, gradient checking, He init, and 25 layers of measured decay

In plain English: mistakes get shared backward, layer by layer

16

The Optimizer Family, One Fix at a Time

κ , momentum, AdaGrad→RMSProp→Adam→AdamW, all built from scratch

In plain English: smarter step sizes get you downhill faster

17

Regularizing Deep Nets, Three Pillars

Dropout, BatchNorm (with backward pass), residual highways, pre-norm

In plain English: guardrails so the network doesn't just memorize

Intuition first, then the mathematics that makes it precise — then code that measures whether the mathematics was telling the truth.

14

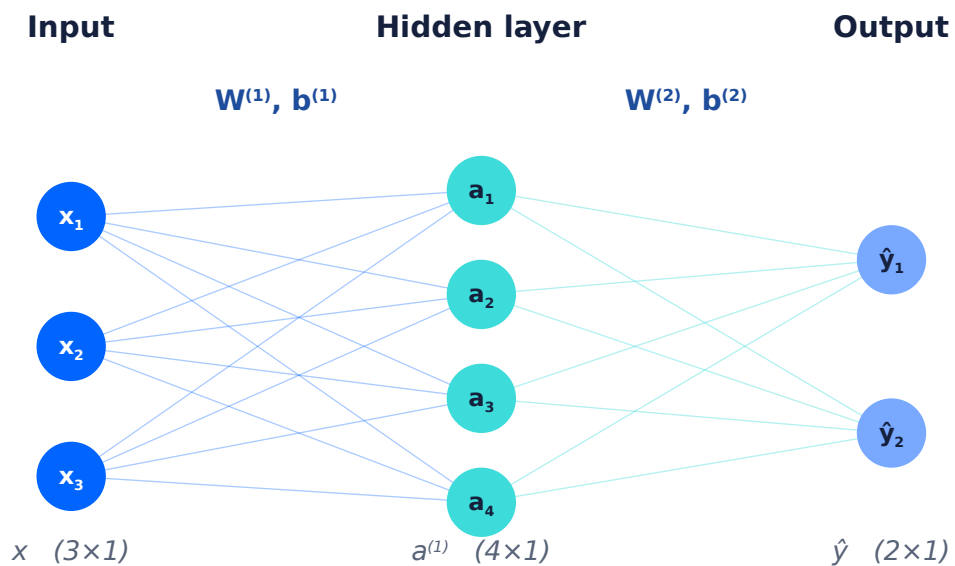
CHAPTER FOURTEEN

The Neuron, and Why Depth Needs a Kink

Strip a neuron to its essence and it is Volume II's linear model with a nonlinear hat on top. Everything interesting about depth comes from that hat.

The simplest possible neural network

Three layers. A handful of neurons. Every idea in this chapter lives in this one picture.



Every arrow carries a number — a weight. Every circle adds up its arrows and decides whether to fire.

FORWARD PROPAGATION — STEP BY STEP

- 1 **One neuron: weigh, sum, add bias**

$$z = w_1x_1 + w_2x_2 + w_3x_3 + b$$

Each input is scaled by its own weight, the results are summed, then bias b shifts the total.

- 2 **The whole hidden layer at once**

$$z^{(1)} = W^{(1)}x + b^{(1)}$$

Same arithmetic for all 4 neurons, stacked into one matrix multiply: $(4 \times 3)(3 \times 1) + (4 \times 1) \rightarrow 4 \times 1$.

- 3 **Apply the activation function**

$$a^{(1)} = \sigma(z^{(1)})$$

σ bends each number. Without this step the whole network collapses into a single straight line.

- 4 **Repeat once more to the output**

$$z^{(2)} = W^{(2)}a^{(1)} + b^{(2)} \rightarrow \hat{y} = \sigma(z^{(2)})$$

One layer's output is the next layer's input: $(2 \times 4)(4 \times 1) \rightarrow 2 \times 1$. $\hat{y}$ is the prediction.

NEURON

A tiny calculator. It multiplies each input by an “importance” weight, adds the results, and decides how strongly to fire.

LAYER

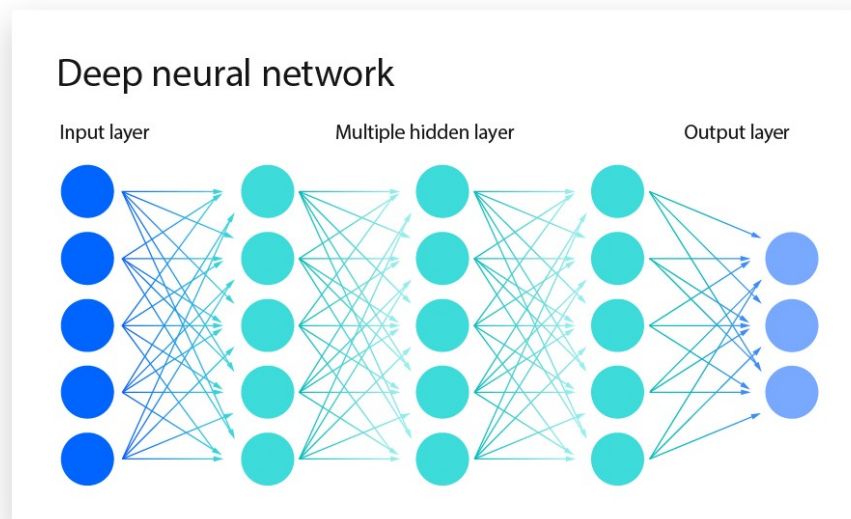
A group of neurons side by side. Each layer takes the previous layer's output as its own input.

WEIGHTS (THE LINES)

Every line is a number. “Training” just means adjusting these numbers until the answers get better.

This is what it looks like in a real model

Same neurons, same layers, same weights as the simple version — just many more of each.



Real networks like GPT or ResNet repeat this exact pattern — just with more neurons per layer, and more layers stacked.

Nothing new here, only scale. A large model can have thousands of neurons per layer and dozens of layers — but it is built from exactly the neuron, layer, and weight ideas you just saw.

Linear regression, wearing a hat

$$\mathbf{a} = \mathbf{f}(\mathbf{w}^T \mathbf{x} + \mathbf{b})$$

the linear model from Volume II

Weigh each input by importance, add them up, decide how strongly to fire

The weighted sum $\mathbf{w}^T \mathbf{x} + \mathbf{b}$ is exactly linear regression — nothing new

The magic is not in one neuron; it is in stacking layers of them

But stacking only works because of that hat, as §14.2 proves

A layer, then a network

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$$

$$\mathbf{a}^{(l)} = \mathbf{f}(\mathbf{z}^{(l)})$$

$\mathbf{f}$ applied elementwise. That pair of equations, repeated L times, is the entire forward pass of a feedforward network — nothing else is happening.

Worth knowing — the bias is not optional

Without $\mathbf{b}$ every boundary is forced through the origin — measured: 0.993 → 0.823 accuracy on data whose boundary is offset. It disappears only before BatchNorm, whose recentring subtracts any constant $\mathbf{b}$ could add — hence `nn.Linear(bias=False)` there.

IN PLAIN ENGLISH — A neuron adds up weighted inputs, tilts the result with a bias, then bends it through a simple curve.

Remove the activations and depth collapses

$$W_2(W_1x) = (W_2W_1)x$$

and by induction over L layers:

$$a^{(L)} = Wx + b$$

$$W = W^{(L)}W^{(L-1)} \dots W^{(1)}$$

Measured, not asserted

$$6.7 \times 10^{-16}$$

max difference between a 50-layer linear net's output and one collapsed
 $Wx + b$

*50 layers · 1,800 parameters → exactly equivalent to
36 numbers and a bias.*

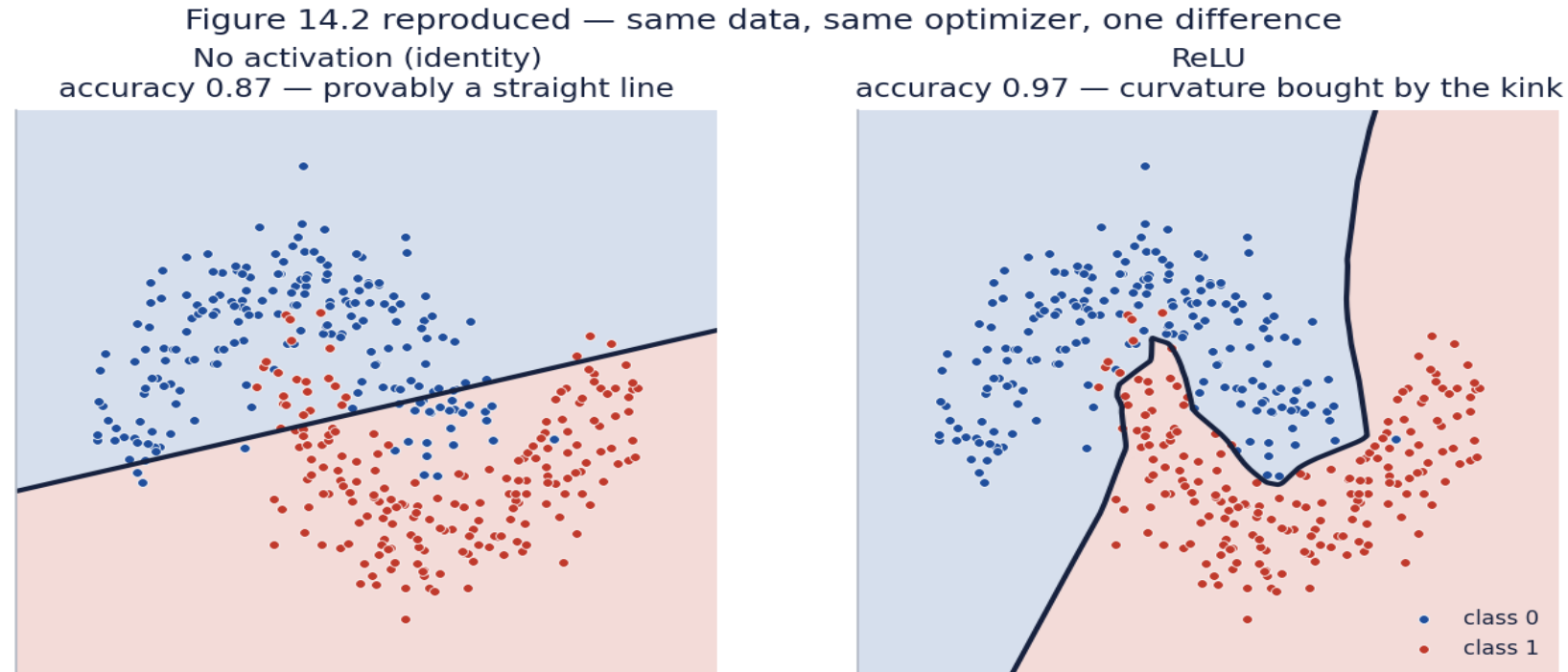
A hundred stacked linear layers equal one linear layer. All that depth — wasted, reduced to a single straight-line model.

The subtlety worth stating in an interview

Depth is not entirely wasted: W is a product of matrices, so narrow layers (width $r \ll d$) force $\text{rank}(W) \leq r$. The network is restricted to low-rank affine maps — a constraint, not extra capacity. The hypothesis class is still linear either way, and that is what matters.

IN PLAIN ENGLISH — *Stacking plain, unbent layers is pointless — multiply straight lines together and you still just get a straight line.*

The curvature is bought entirely by the kink



0.87

accuracy, no activation

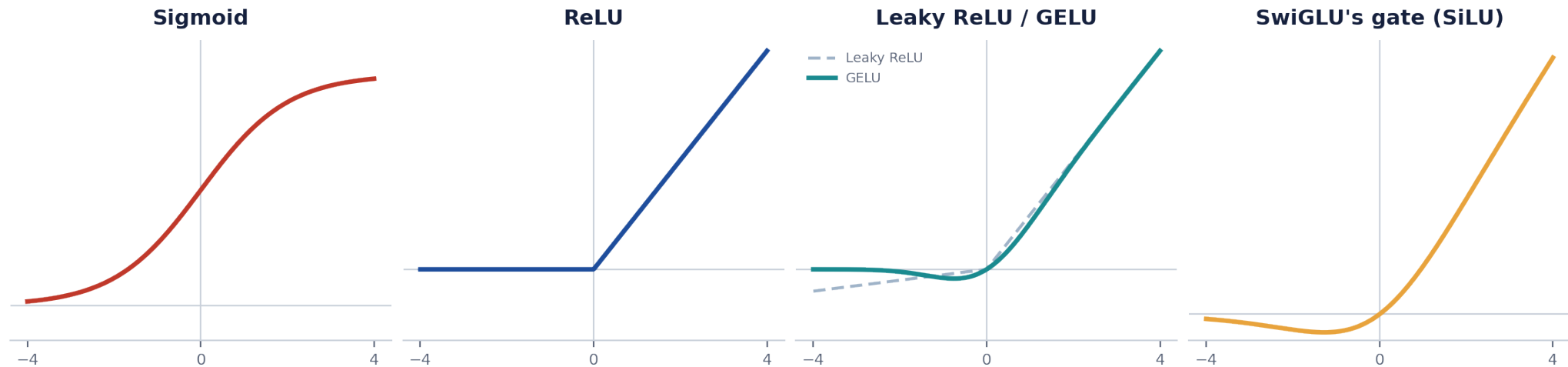
0.97

accuracy, ReLU

The left panel is not undertrained — it is at the global optimum of its hypothesis class, reached with 2,000 wasted parameters.

IN PLAIN ENGLISH — Adding that one bend is what lets the network draw a curved boundary instead of a straight one.

A story of fixing one problem after another



The arc: sigmoid → ReLU (fix saturation) → Leaky / GELU (fix dying + smoothness) → SwiGLU (gate it)

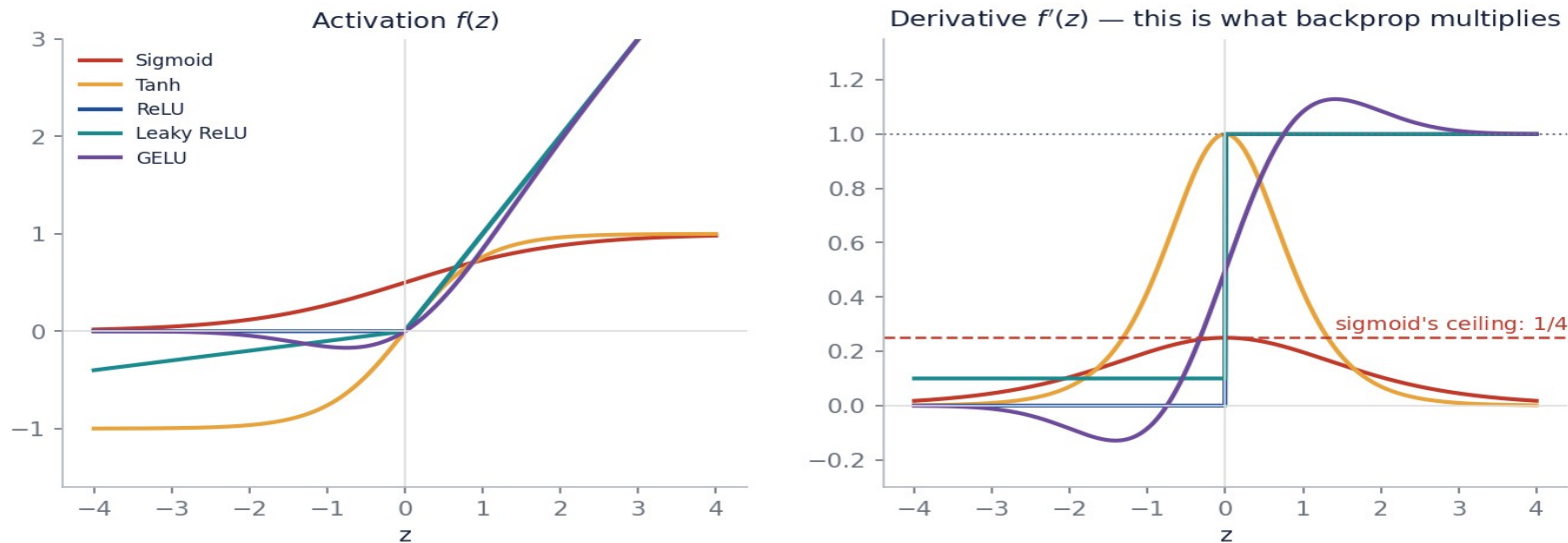
GELU is a stochastic idea in disguise

$z \cdot \Phi(z)$ is the expected value of a gate that keeps z with probability $\Phi(z)$ and zeroes it otherwise — dropout's coin-flip made deterministic and input-dependent. Swish/SiLU is the same idea with a logistic gate.

§14.3 DERIVATION

Sigmoid's gradient cannot exceed $\frac{1}{4}$ — proven, then measured

Figure 14.3 reproduced — sigmoid's gradient literally cannot reach the dashed line



$$\sigma'(z) = \sigma(1 - \sigma)$$

$$g(s) = s - s^2$$

$$g'(s) = 1 - 2s = 0$$

$$\Rightarrow s^* = \frac{1}{2}$$

$$g''(s) = -2 < 0$$

$$\Rightarrow \text{global max} = \frac{1}{4}$$

Real curves and their exact derivatives. The dashed line is the $\frac{1}{4}$ ceiling — sigmoid literally cannot reach it.

$$\left(\frac{1}{4}\right)^5$$

$$\approx 10^{-3}$$

$$\left(\frac{1}{4}\right)^{10}$$

$$\approx 10^{-6}$$

$$\left(\frac{1}{4}\right)^{25}$$

$$\approx 10^{-15}$$

Not "might vanish" — cannot avoid shrinking by at least a million-fold across ten layers, before the weights even get a say.

IN PLAIN ENGLISH — Sigmoid shrinks the signal by at least 75% per layer — stack enough layers and it vanishes completely.

Dead ReLUs are a learning-rate symptom, not a curse

1 — TOO LARGE A STEP

One oversized gradient update pushes a neuron's bias sharply negative.

2 — ALWAYS OFF

Every input now gives a negative pre-activation, so ReLU outputs exactly 0.

3 — NO WAY BACK

Zero output means zero gradient, too — the neuron gets no signal to recover.

Two different numbers

Zero-activation rate near 50% is healthy sparsity. A permanently dead fraction climbing past 90% is the pathology. A unit is only dead if it is inactive for ALL inputs — so this is a per-dataset property.

This chain reaction is what “ReLU death” really is — not mysterious, just arithmetic.

Cause: one large step drives the bias strongly negative; the unit then outputs 0 for every input and never recovers

First fix is not Leaky ReLU — it is lower the learning rate, or add warmup (§16.7)

Diagnose by logging the fraction of units with zero activation across a validation batch

Leaky ReLU's actual contribution: $f' = \alpha > 0$ on the negative side, so a dead unit still receives signal and can climb back

"What activation would you use in a new Transformer today?"

SiLU / Swish

$$z \cdot \sigma(\beta z)$$

z multiplied by a soft keep-probability, with a logistic gate.

GELU

$$z \cdot \Phi(z)$$

Near-identical curve, with the normal CDF as the gate. BERT and GPT.

SwiGLU

$$(\text{SiLU}(xW_1)) \odot (xW_2)$$

The feed-forward block splits into two projections, one gating the other. Llama, PaLM, most recent LLMs.

SwiGLU costs a third projection but reliably lowers loss, which is why the extra parameters are considered worth it.

Chapter 14 in one breath

A neuron is linear regression plus a nonlinear kink, and that kink is load-bearing: without it L layers provably collapse to one affine map, so depth buys only a rank constraint. Sigmoid's derivative is capped at exactly $\frac{1}{4}$, which dooms deep stacks; ReLU's slope of 1 removes that shrinkage and made modern depth trainable.

IN PLAIN ENGLISH — Modern activations softly “gate” how much of a signal gets through, instead of switching it fully on or off.

15

CHAPTER FIFTEEN

Backpropagation, and the Vanishing Signal

Two results that turn out to be the same equation read twice: the recursion that makes backprop fast is the recursion that makes the gradient die.

The whole field lives in the gap between $O(P)$ and $O(P^2)$

Forward pass

$O(P)$

one sweep, caching every z and a

Backward pass

$O(P)$

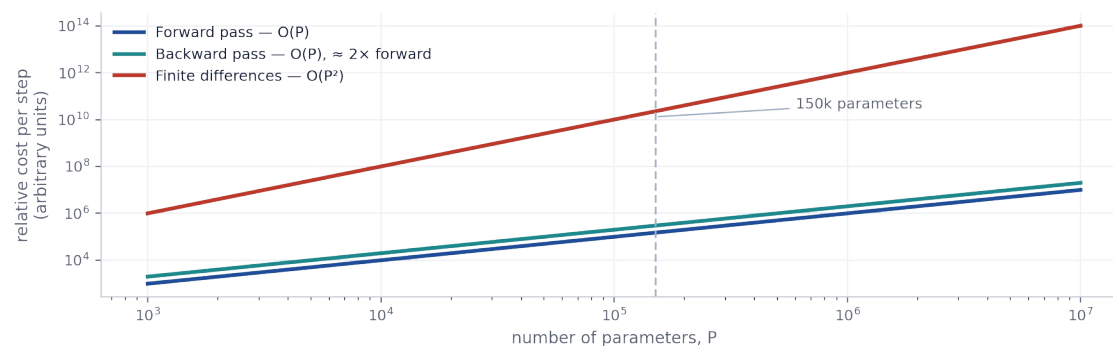
$\approx 2\times$ forward: each layer computes $\partial L / \partial W$ and the δ to pass down

Finite differences

$O(P^2)$

one forward pass per parameter, plus cancellation error

The $O(P)$ vs $O(P^2)$ gap, plotted across model sizes



At 150k parameters finite differences already costs five orders of magnitude more per step — and real models are a thousand times larger again.

One recursion, and it is the whole chapter

$$\delta^{(l)} \equiv \partial L / \partial z^{(l)}$$

$$\delta^{(l)} = ((W^{(l+1)})^T \delta^{(l+1)}) \odot f'(z^{(l)})$$

$$\partial L / \partial W^{(l)} = a^{(l-1)} \delta^{(l)T} \quad \# \text{ outer product}$$

$$\partial L / \partial b^{(l)} = \delta^{(l)}$$

The transpose appears because we sum over the first index of W

The $\odot$ appears because f acts elementwise, so $\partial a / \partial z$ is diagonal

Two factors, applied multiplicatively, once per layer — remember that

Computing blame for layer 2 reuses the δ already computed for layer 3

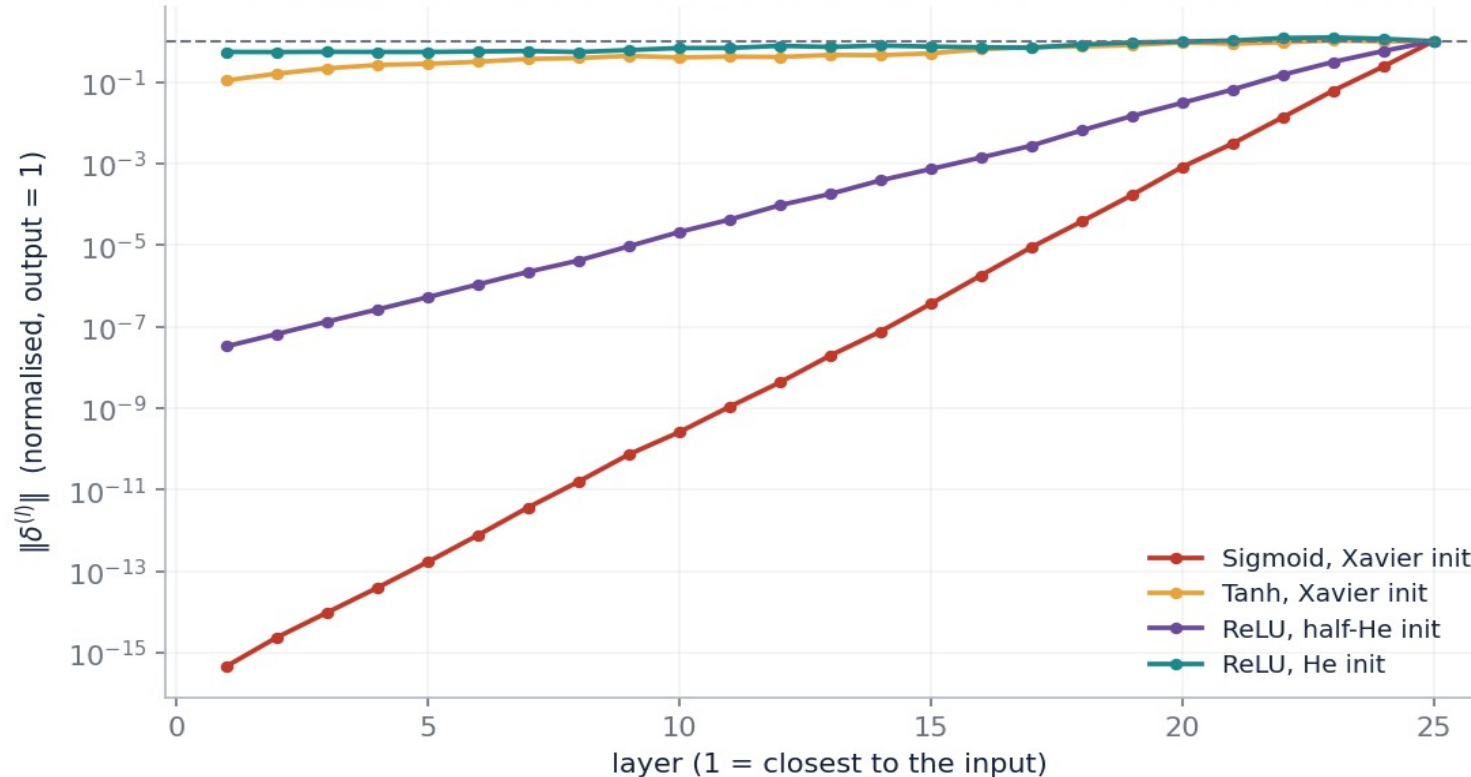
Backprop is dynamic programming on the computation graph

That reuse of intermediate results — not the chain rule itself — is what makes backprop fast. It is memoization, the same idea from your CS prep. Naming the connection out loud signals real understanding. And it is one instance of reverse-mode autodiff: reverse mode computes one output's gradient w.r.t. many inputs in a single sweep, and deep learning has exactly one scalar output and millions of inputs, so it wins by a factor of P .

IN PLAIN ENGLISH — *To find out how wrong each layer was, take the next layer's blame and pass it backward through that layer's own math.*

Straight lines on a log axis: exponential decay, as derived

Figure 15.2 reproduced — gradient norms through a real 25-layer network



Measured through a real 25-layer network. Sigmoid loses ~15 orders of magnitude by layer 1; ReLU + He sits flat at 1.0; half-He shows what a factor-of-2 init error costs.

$$\|\delta^{(0)}\| \leq (\gamma \cdot \sigma_{\max})^{L-1} \|\delta^{(L)}\|$$

$$\gamma = \sup |f'(z)|$$

$\sigma_{\max}$ = largest singular value of W

Sigmoid: $\gamma = 1/4$

needs $\sigma_{\max} \approx 4$ in every layer — enormous weights, which push z where $|\sigma'|$ collapses far below $1/4$. Squeezed between vanishing and saturation, with no escape.

ReLU: $\gamma = 1$

the condition reduces to $\sigma_{\max} \approx 1$ — achievable by initialization alone.

IN PLAIN ENGLISH — Multiply enough numbers under one together and the product shrinks to zero — the vanishing gradient, in one line.

The $\frac{1}{2}$ that ReLU introduces is the whole difference

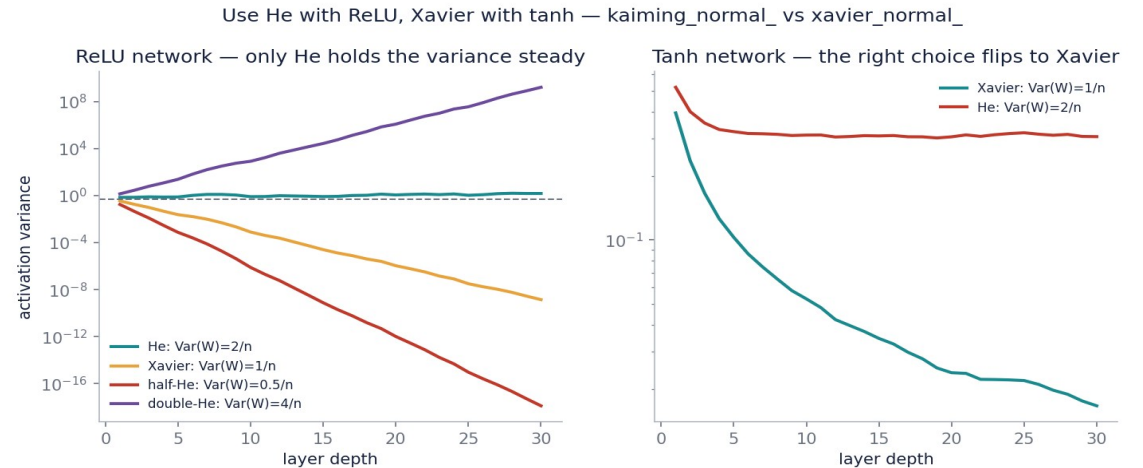
$\text{Var}(z^{(l)}) = n \cdot \text{Var}(W) \cdot \frac{1}{2} \cdot \text{Var}(z^{(l-1)})$
 require $n \cdot \text{Var}(W) \cdot \frac{1}{2} = 1$
He $\text{Var}(W) = 2 / n_{\text{in}}$
Xavier $\text{Var}(W) = 2 / (n_{\text{in}} + n_{\text{out}})$

Why exactly one half?

If z is symmetric about 0 then z^2 is identical on both halves — but ReLU zeroes one of them, so exactly half the second moment survives.

Verified on 2 million samples: $E[a^2]/E[z^2] = 0.5008$.

Note it is the second moment $E[a^2]$, not $\text{Var}(a)$ — they differ whenever a has nonzero mean, which ReLU guarantees.



Activation variance through 30 layers. Use He with ReLU, Xavier with tanh — kaiming_normal_vs_xavier_normal_.

"Why can't we just initialize all weights to zero?"

$\text{Var}(W)=0$ gives $\text{Var}(z)=0$, and every unit in a layer then receives an identical δ — so identical updates, and the units stay clones forever. A 512-unit layer is permanently a 1-unit layer. Any constant init has the same fate. Biases can safely start at zero: the weights have already broken the symmetry.

IN PLAIN ENGLISH — Get the starting “volume” of the weights wrong and the signal drifts to silence or noise before training even begins.

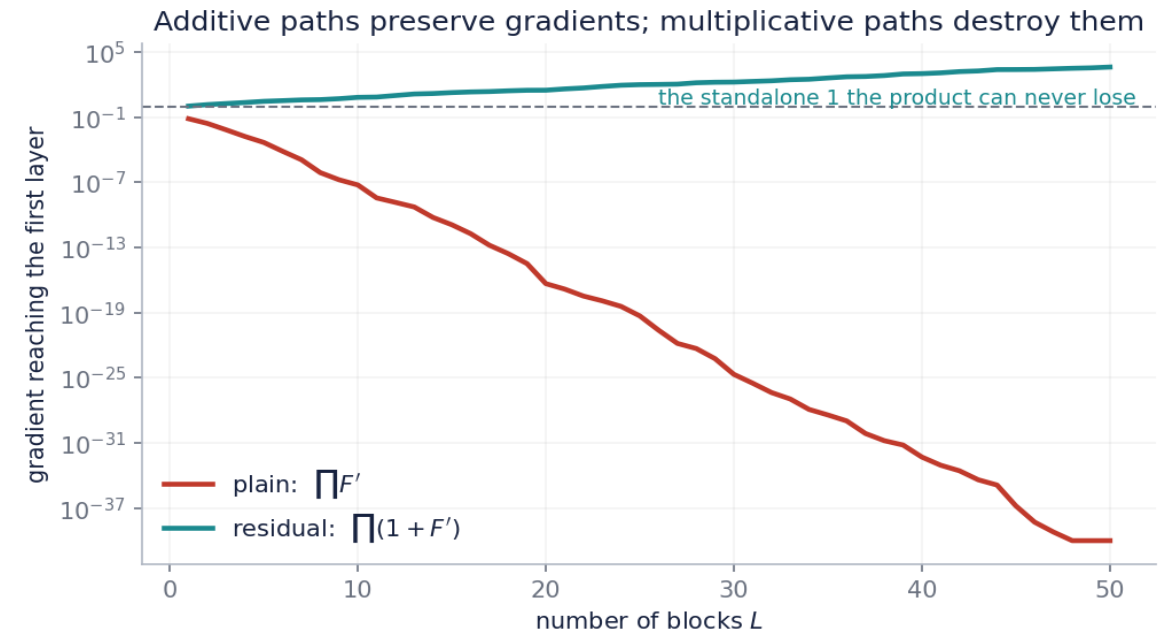
Additive paths preserve gradients; multiplicative paths destroy them

plain $y = F(x)$ $\partial y / \partial x = F'(x)$
 residual $y = x + F(x)$ $\partial y / \partial x = 1 + F'(x)$

The product over L residual blocks contains a term that is identically 1. The gradient can always get home.

Two architectures, eighteen years apart

LSTM's cell state (1997) and ResNet's skip connection (2015) exploit the identical trick: give the gradient an additive highway to flow backward on, bypassing the multiplicative shrinkage. §17.4 measures this through 50 real layers.



Same F' values in both curves. After 50 blocks the plain chain is at 10^{-40} ; the residual chain is above 1.

IN PLAIN ENGLISH — Give the gradient a shortcut that skips the multiplication entirely, and it can't vanish on the way back.

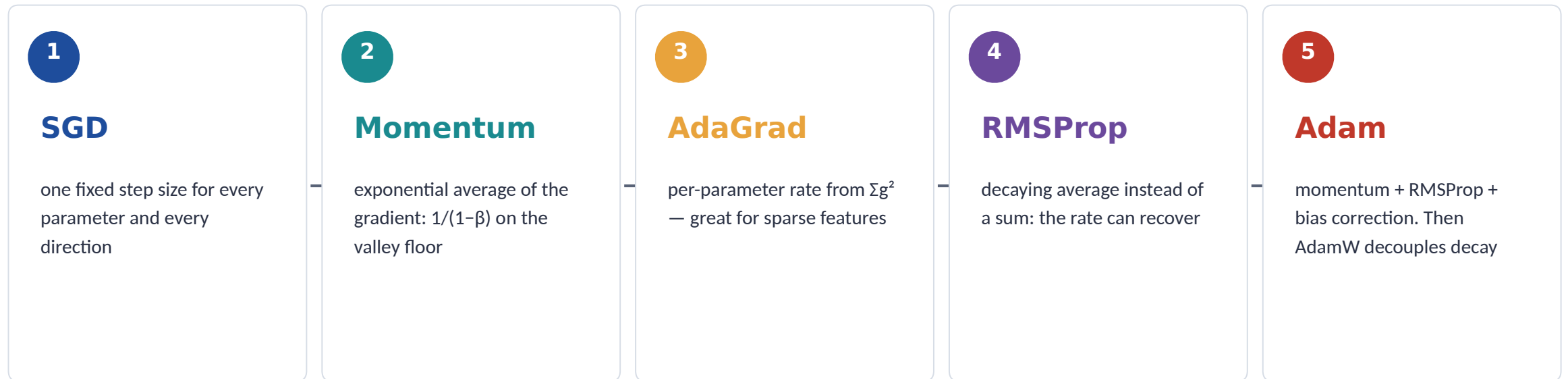
16

CHAPTER SIXTEEN

The Optimizer Family, One Fix at a Time

Not a menu of unrelated choices — a lineage. Read the arrows as complaints, not chronology: each optimizer exists because someone identified one concrete failure in the last one.

You can reconstruct the whole family from first principles



Every optimizer here is the same skeleton: a step downhill, scaled by something built from gradient history.

The complaint that produced each arrow

SGD wobbles across ravines → Momentum. One global step size suits neither frequent nor rare features → AdaGrad. AdaGrad's rate decays to zero and can never recover → RMSProp. Take the best of both → Adam. L2 gets rescaled by v → AdamW.

IN PLAIN ENGLISH — Every optimizer here is the same idea — take a step downhill — dressed up with a smarter way to size that step.

SGD's step is set by the steepest direction, its progress by the flattest

$L(\theta) = \frac{1}{2} \theta^T H \theta$, eigenvalues $\lambda_{\min} \dots \lambda_{\max}$

$u_i^{(t+1)} = (1 - \eta \lambda_i) u_i^{(t)}$

stability $\Rightarrow \eta < 2/\lambda_{\max}$

$|r_{\min}| > |1 - 2/\kappa|$, $\kappa = \lambda_{\max}/\lambda_{\min}$

$\kappa = 100$

2.0%

best-case progress per
step
along the flat direction

$O(\kappa)$

≈ 481

SGD steps to converge
at $\kappa = 100$ (measured)

$O(\sqrt{\kappa})$

≈ 101

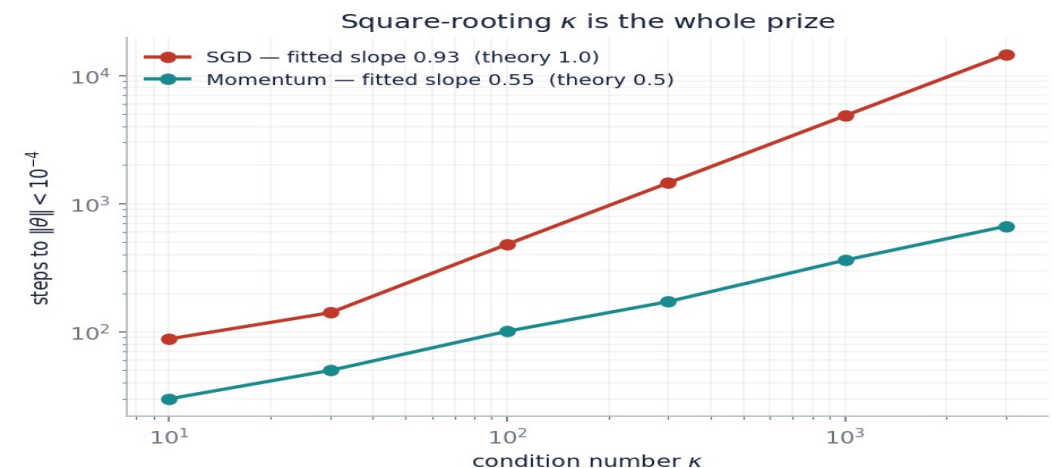
momentum steps for
the
same tolerance
(measured)

Deep nets routinely have κ in the thousands

Momentum square-roots κ — that is the whole prize, and it is the same rate conjugate gradient achieves

AdaGrad / RMSProp / Adam attack κ itself, rescaling each coordinate so the effective spectrum is flatter

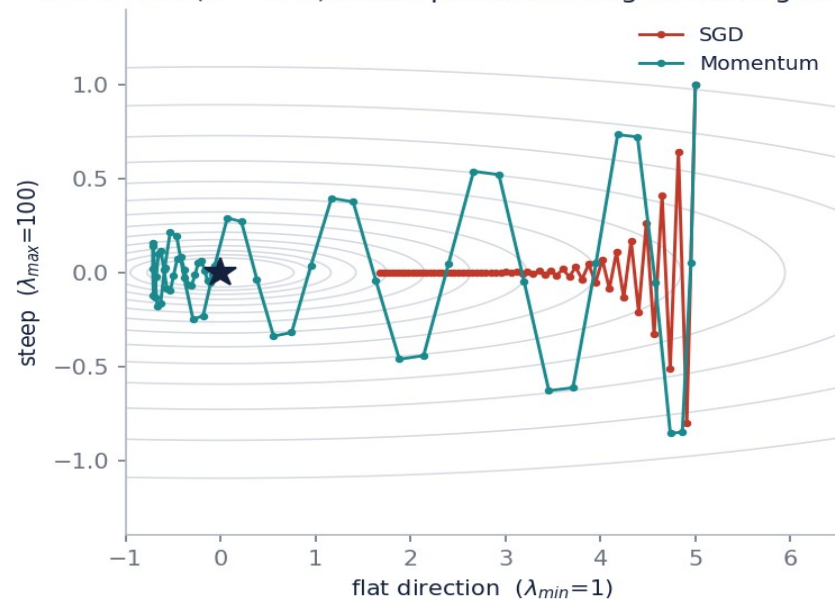
Fitted slopes: 0.9 for SGD, 0.5 for momentum



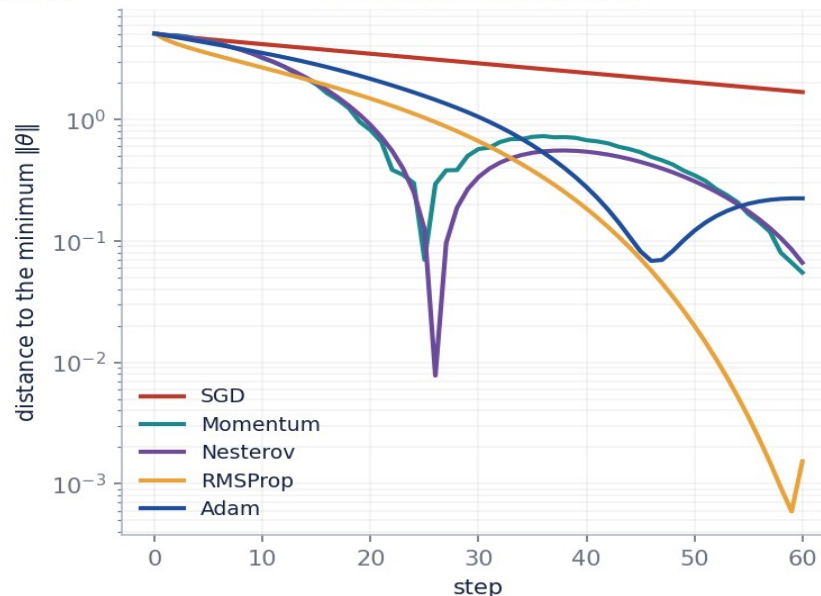
IN PLAIN ENGLISH — One step size can't suit a steep direction and a flat one at once — progress crawls at the flat direction's pace.

Every method tuned to ~90% of its own stability limit — no strawmen

The ravine ($\kappa = 100$): SGD spends its budget crossing the walls



The honest scoreboard



Why it works

consistent direction

$$\times 1/(1-\beta) = 10$$

oscillating direction

$$\times 1/(1+\beta) \approx 1/2$$

= a 19× relative preference for the valley floor

A real quadratic with $\kappa = 100$. Left: SGD spends its budget crossing the walls. Right: the honest scoreboard.

The stability caveat worth volunteering

Because the effective step grows by $1/(1-\beta)$, momentum's stable learning rate is smaller: the heavy-ball condition is $\eta(1+\beta) < 2/\lambda_{\max}$. It still wins decisively, because it converges in $O(\sqrt{\kappa})$ steps rather than $O(\kappa)$. Nesterov refines it by evaluating the gradient at the look-ahead point, so the ball can brake before it overshoots rather than after.

IN PLAIN ENGLISH — Momentum remembers which way it's been heading, so it barrels through slow, flat stretches instead of creeping across them.

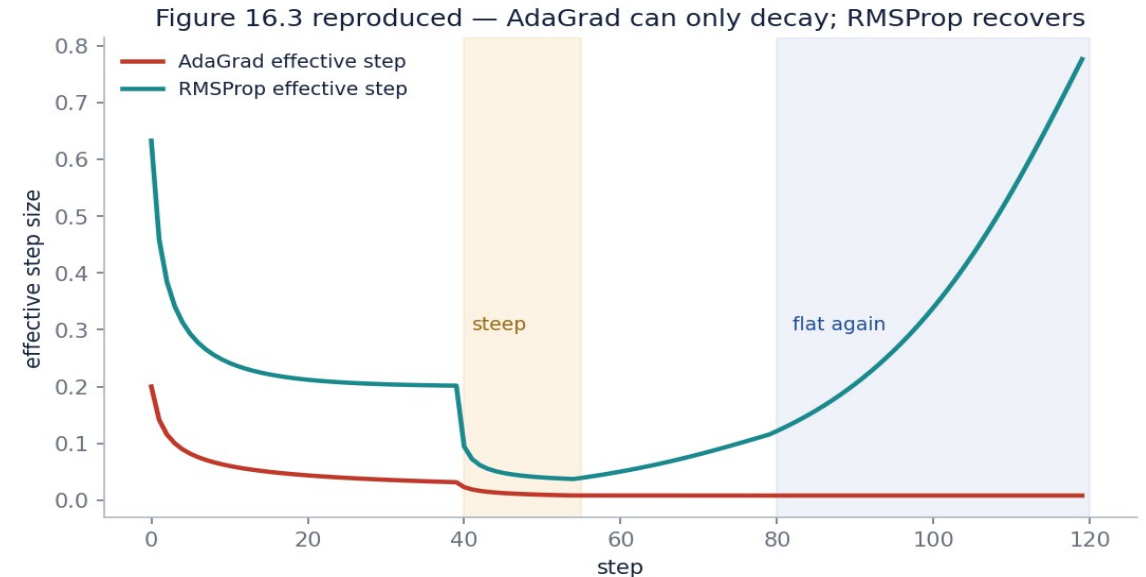
One character changes the story: a sum becomes an average

AdaGrad $G_t = \sum g_k^2$ (a SUM)
RMSProp $v_t = \beta v_{t-1} + (1-\beta)g_t^2$ (an AVERAGE)
 $\theta_{t+1} = \theta_t - \eta/(\sqrt{\cdot} + \epsilon) \odot g_t$

The failure in one line

If gradients have roughly constant scale g , then $G_t \approx tg^2$, so the effective rate decays like $\eta/(g\sqrt{t})$ — a $1/\sqrt{t}$ schedule you never chose and cannot switch off.

That is provably optimal for convex problems, which is what AdaGrad was designed for. It is wrong for non-convex deep nets, where the landscape you face at step 10,000 has nothing to do with the gradients from step 10.



The same gradient stream fed to both accumulators. AdaGrad's step decays monotonically by construction — the curve can never turn back up.

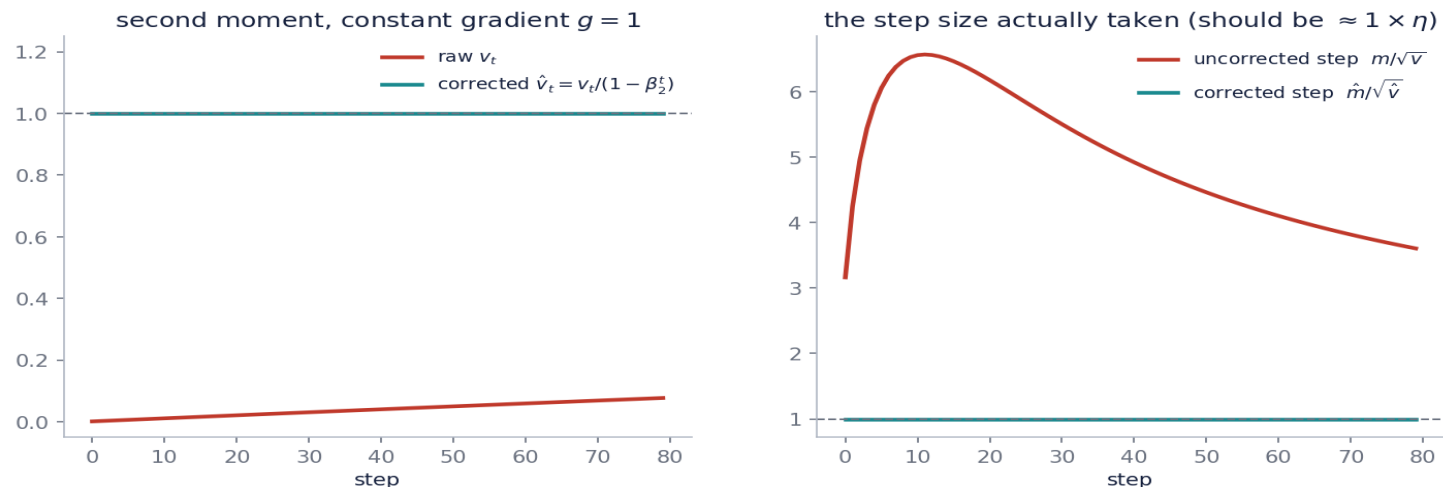
What dividing by $\sqrt{v}$ actually buys: unit-free steps

$g/\sqrt{\hat{v}}$ is dimensionless — exactly 1.0000 whether the gradients are 10^{-7} or 10^3 . That is why $\eta = 10^{-3}$ is a sane Adam default across wildly different architectures. And ϵ 's real job is to cap the maximum step at η/ϵ .

IN PLAIN ENGLISH — Give each weight its own step size based on how much it's moved before — and let that memory fade so it can keep learning.

Where $1/(1-\beta^t)$ comes from, and why it fades

Figure 16.4 reproduced — bias correction is not optional



A constant gradient of 1 fed to both accumulators. The asymmetry between the curves is why bias correction is not optional.

The exact size of the lie

$$\mathbf{E}[\mathbf{m}_t] = (1 - \beta_1^t) \mathbf{E}[\mathbf{g}]$$

$$\hat{\mathbf{m}}_t = \mathbf{m}_t / (1 - \beta_1^t) = \mathbf{E}[\mathbf{g}] \quad \checkmark$$

$\mathbf{m}_0 = \mathbf{v}_0 = 0$ is not a neutral choice — it is a data point claiming "the gradient is zero," and it drags the average down for many steps. Dividing by $(1 - \beta^t)$ is the exact reciprocal of that bias.

1000×

how badly v_1 underestimates
with $\beta_2 = 0.999$

3.16×

the first step you would take
uncorrected, vs the intended η

1.00×

the corrected first step —
honest from $t = 1$

IN PLAIN ENGLISH — Adam blends momentum's direction with RMSProp's step-sizing, correcting both for an early bias at training's start.

AdamW: the single most cited subtle optimizer bug

$$\tilde{L} = L + (\lambda/2)\|\theta\|^2 \Rightarrow g_t = \nabla L + \lambda\theta_t$$

Adam divides EVERYTHING by $\sqrt{v}$:

effective decay on $\theta_i = \eta\lambda / (\sqrt{v_{t,i}} + \epsilon)$

AdamW applies decay OUTSIDE that

Under Adam the quiet parameter is decayed $\sim 100\times$ harder than the noisy one — for the same λ . That is backwards from the intent, and λ 's meaning drifts during training.

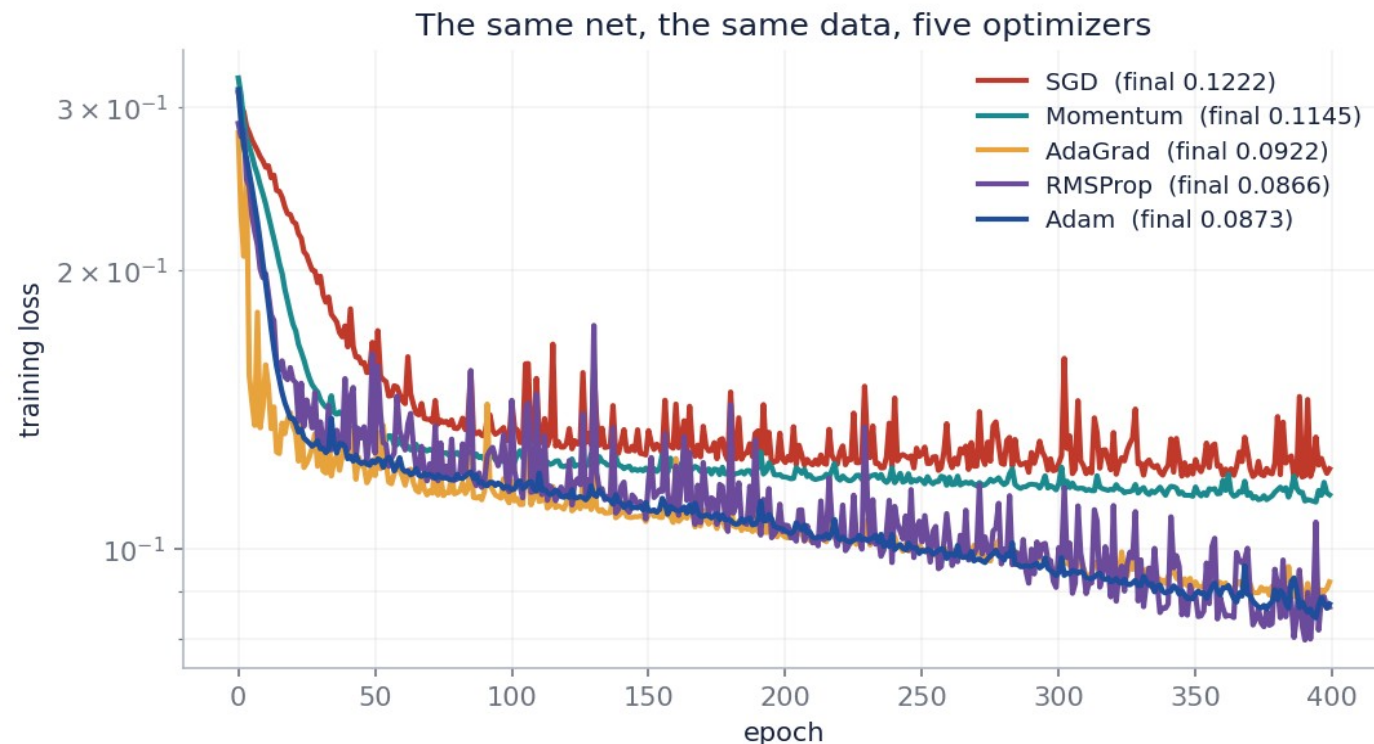
For plain SGD, L2 and weight decay are algebraically identical — which is why the field conflated them for years. Under any adaptive optimizer they are not.

```
torch.optim.Adam (weight_decay= $\lambda$ ) # broken  
torch.optim.AdamW(weight_decay= $\lambda$ ) # decoupled
```

Schedules on top

Warmup: early $\hat{V}$ is a high-variance estimate from few gradients (relative spread 0.44 at step 10 vs 0.02 at step 4000), and at init the surface is worst-conditioned. Cosine decay: shrinking η lowers the noise floor the iterate settles at. Large η explores, small η refines.

Same net, same data, five optimizers built from scratch



Read the ordering, don't declare a winner

Momentum beats plain SGD, as the ravine analysis says it must.

AdaGrad drops fast early — then visibly flattens. That is the $1/\sqrt{t}$ decay you never chose, on a real problem.

RMSProp and Adam replaced that sum with a decaying average, keep making progress, and finish lowest.

"Adam is momentum's smoothed-direction memory plus RMSProp's per-parameter step-size memory, with bias correction because both start at zero."

When Adam is the wrong default: well-tuned SGD+momentum often generalizes better on large-scale vision · AMSGrad's counterexample shows Adam can fail to converge even on convex problems · m and v triple optimizer state (84 GB for a 7B model in fp32)

17

CHAPTER SEVENTEEN

Regularizing Deep Nets, Three Pillars

Dropout, batch normalization, residual connections. Two of the three are really answers to Chapter 15's vanishing-gradient problem wearing a regularization costume.

Why the $1/(1-p)$ factor is there

Prevents co-adaptation: if any neuron might vanish, none can rely on a specific partner being present

A stealth ensemble: each step samples a different sub-network — 2^n of them sharing one set of weights

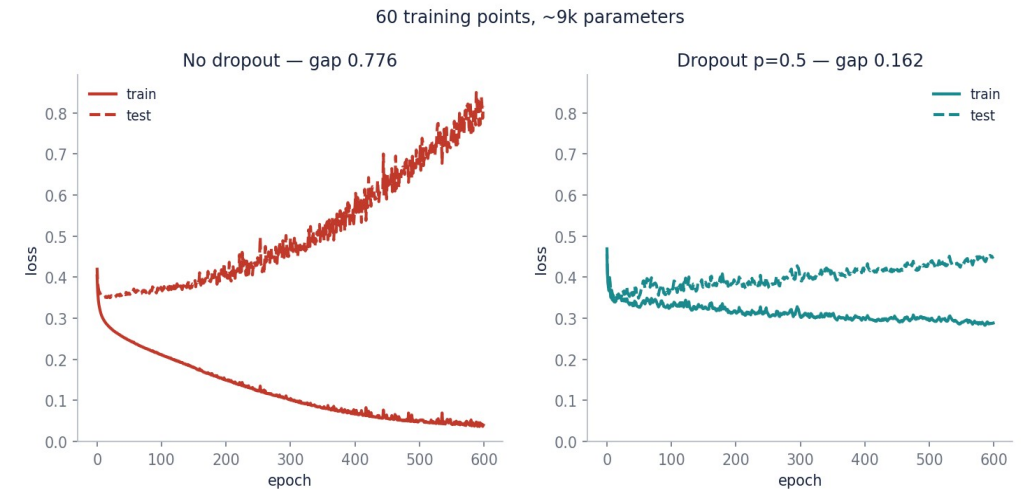
Train multiplies a by $r \sim \text{Bernoulli}(1-p)$, so $E[a] = (1-p)a$; inference sees a

Left unrepaired, every downstream pre-activation is inflated by $1/(1-p)$ at test time — $2\times$ at $p = 0.5$

(a) scale at test: $a \leftarrow (1-p)a$

(b) scale at train: $a \leftarrow ra/(1-p)$ ← inverted dropout

Frameworks all pick (b), because it keeps inference identical to a network that never had dropout — `model.eval()` just switches the mask off.



60 training points, ~4,900 parameters. Test loss 0.82 → 0.45; test accuracy 0.83 → 0.86.

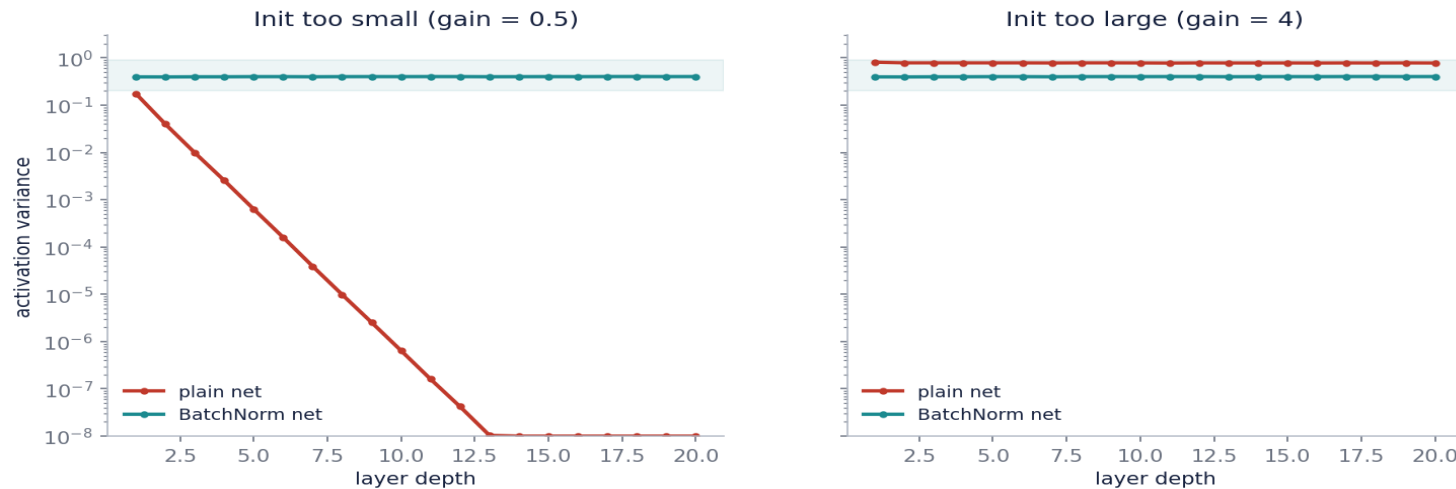
The caveat that makes it an approximation

Matching expectations is not the same as averaging the sub-networks' predictions: $E[f(x)] \neq f(E[x])$ for nonlinear f . Test-time dropout is a first-order approximation, exact only for linear models. MC dropout — sampling many masks and averaging — is the honest version.

IN PLAIN ENGLISH — Randomly switch off some neurons during training so the rest can't just lean on a few reliable teammates.

Normalization proposes; the network disposes

Figure 17.2 reproduced — BatchNorm pins every layer to a healthy band



Activation variance at each depth of a 20-layer tanh net. BatchNorm pins every layer to the same healthy band regardless of the init.

The train/test asymmetry — a classic interview trap

Training uses the current mini-batch's μ and σ ; test time uses a running average, because there may be no batch and predictions must not depend on whoever else was in it. Measured: in `train()` mode a batch of size 1 normalizes to exactly zero, and the same example gets different predictions in different batches. That is why `model.eval()` is mandatory.

γ and β matter more than they look

Without them BatchNorm would force every pre-activation to zero-mean unit-variance — which for a sigmoid or tanh pins the unit to the near-linear part of its curve and destroys exactly the expressiveness §14.2 fought for. With them, the network can learn $\gamma = \sigma$, $\beta = \mu$ and undo the normalization entirely.

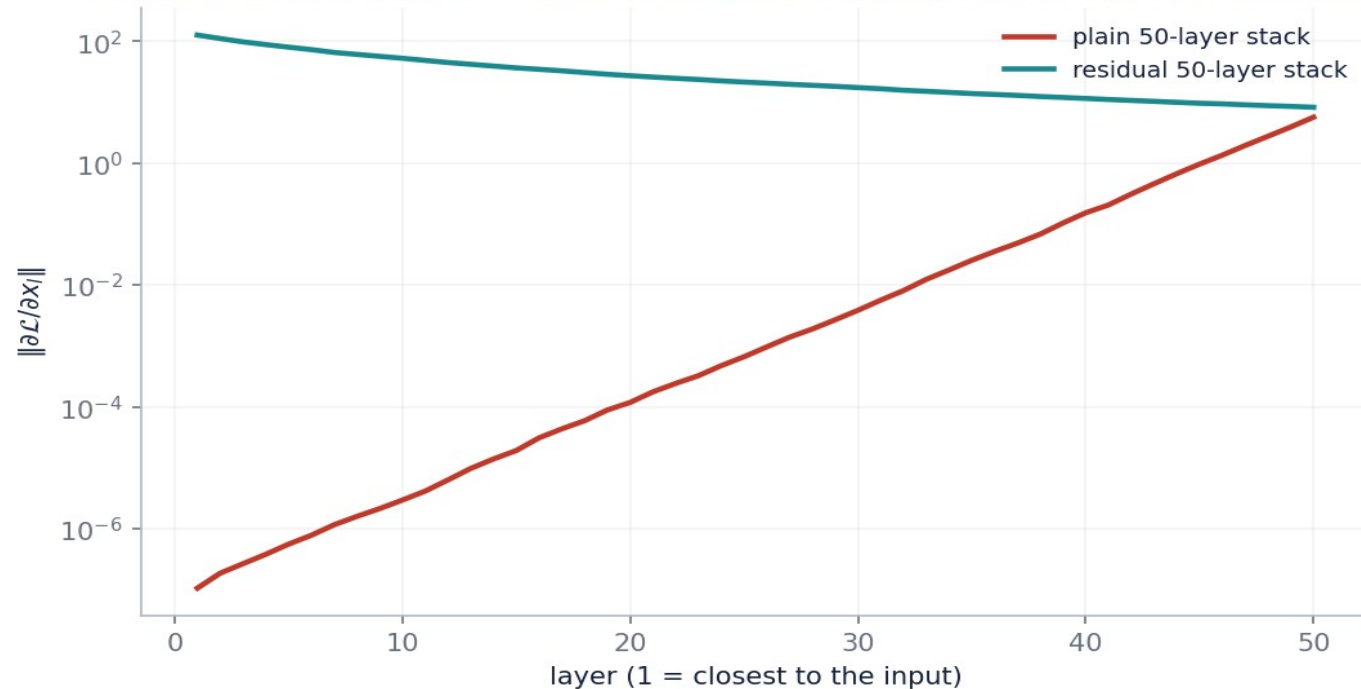
"Internal covariate shift" is probably the wrong explanation

Santurkar et al. (2018) injected deliberate distributional noise after BatchNorm and training still improved. The better-supported mechanism is landscape smoothing — improving the loss's Lipschitz constants — which is what permits large learning rates.

IN PLAIN ENGLISH — Re-center and re-scale the signal at every layer so it stays in a healthy range no matter how deep the network gets.

The gradient highway, measured through 50 layers

Figure 17.4 reproduced — identical weights, differing only by the skip connections



$$\frac{\partial L}{\partial x_1} = \frac{\partial L}{\partial x_L} \cdot \prod (I + J_k)$$

$$\prod (I + J_k) = I + \sum J_k + \sum J_k J_j + \dots$$

The subset of size zero contributes exactly I — a path with derivative 1 from the loss to any layer.

$10^9 \times$

more gradient at layer 1, identical weights

Read the guarantee carefully

In a plain network, if the J 's are small the gradient is gone — the product of small things is smaller. In a residual network the worst case is the identity: the deep net degenerates gracefully into a shallower one rather than into silence. This also answers "why is it easier to learn a residual?" — if the optimal map is near-identity, F only has to learn ≈ 0 .

IN PLAIN ENGLISH — Each layer learns a small adjustment on top of its input, instead of the whole transformation from scratch.

Where you put the LayerNorm decides if the highway is clean

Post-norm (the original Transformer)

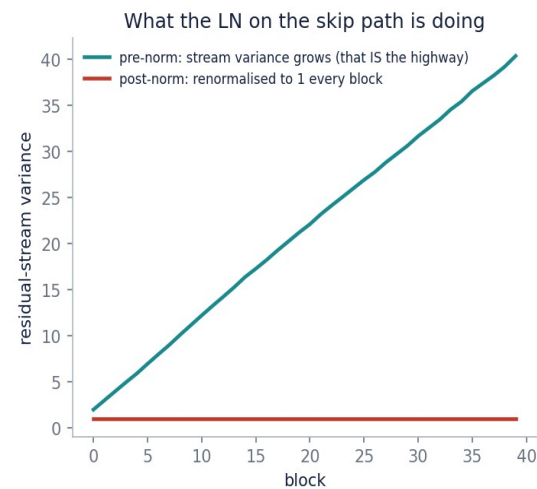
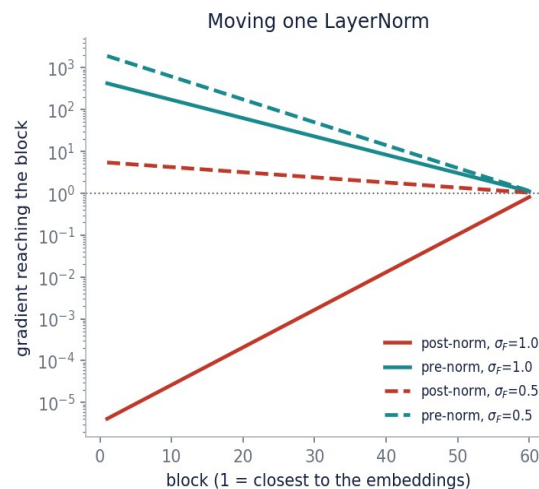
$$x \leftarrow \text{LayerNorm}(x + F(x))$$

The normalization sits ON the skip path, so the clean I is no longer clean. Deep post-norm stacks need careful warmup to train at all.

Pre-norm (modern LLMs)

$$x \leftarrow x + F(\text{LayerNorm}(x))$$

The identity path is completely unobstructed from the loss all the way to the embeddings — and it trains far more robustly at depth.



Post-norm pays a factor $1/\sqrt{1+\sigma^2 F}$ on the skip path every block, and that compounds.

If you can explain why moving one LayerNorm makes 100-layer Transformers trainable, you have understood this entire chapter.

IN PLAIN ENGLISH — Where you place “normalize” relative to the shortcut decides if that shortcut stays a clean, unobstructed highway.

Rapid fire — and where the answer lives

Why do we need nonlinear activations?

§14.2 · NB1

$W_2(W_1x) = (W_2W_1)x$; any depth collapses to one affine map. Follow-up: depth still constrains rank.

Why did ReLU replace sigmoid?

§14.3 · NB1

$\sigma' \leq \frac{1}{4}$ at its single best point, so L layers scale by $\leq (\frac{1}{4})^L$. ReLU's slope is exactly 1.

Why is backprop better than numerical gradients?

§15.2 · NB2

Finite differences is $O(p^2)$ and adds cancellation error. Backprop reuses each δ — memoization on the graph.

Why can't we initialize all weights to zero?

§15.4 · NB2

Every unit receives an identical δ , so identical updates — a 512-unit layer is permanently a 1-unit layer.

He vs Xavier — what's the difference?

§15.4 · NB2

Exactly the $\frac{1}{2}$ that ReLU introduces by discarding the negative half of the second moment.

Why does momentum converge in $O(\sqrt{\kappa})$?

§16.2 · NB3

It turns a first-order recursion into a heavy-ball one whose slow mode contracts like $1 - 2/\sqrt{\kappa}$.

What does Adam's bias correction solve?

§16.6 · NB3

$m_0 = v_0 = 0$ shrinks both estimates early; $1/(1-\beta^t)$ is the exact reciprocal, and it fades on its own.

Why does dropout behave differently at test time?

§17.2 · NB4

$E[ra] = (1-p)a$, so inference is inflated by $1/(1-p)$. Inverted dropout divides during training instead.

Trains well, garbage at inference — where first?

§17.3 · NB4

`model.eval()`. BatchNorm switches to running averages; dropout switches its mask off.

How do residuals help vanishing gradients?

§17.4 · NB4

$\Pi(I + J_k)$ always contains a standalone I — a path with derivative exactly 1 from the loss to any layer.

Each of these is answered by a section you have already derived — the point is to reconstruct, not recite.

THE SINGLE THREAD

Which part of the multiplicative chain is this protecting?

Nearly every technique in Volume V is a response to one problem: a gradient signal, multiplied layer after layer, either dies or explodes. ReLU protects D . He init protects W . BatchNorm keeps z where D is healthy. Residual connections sidestep the product. Clipping tames the exploding half. Adam reshapes the step once the gradient arrives.

01

From Linear Regression to Neurons

02

Backpropagation & Vanishing
Gradients

03

The Optimizer Family

04

Regularizing Deep Nets